



ค่ายโอลิมปิกวิชาการ วิชาคอมพิวเตอร์

POINTER

ผศ.ดร.เจียบวุฒิ รัตนวิไลสกุล chaibwoot.r@sci.kmutnb.ac.th

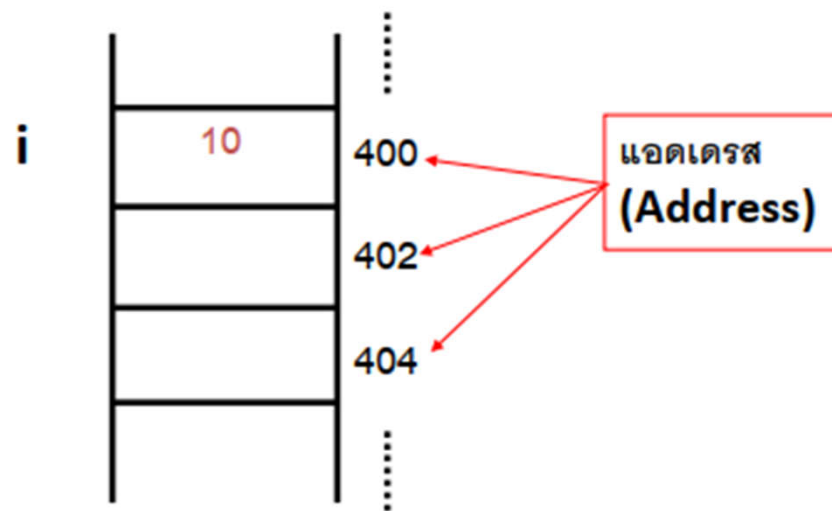
ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

พอยน์เตอร์

- เป็นชนิดข้อมูลชนิดหนึ่งของภาษา C (int, float, char)
 - **มีความเร็วในการทำงานสูง**
 - ช่วยประหยัดเนื้อที่ในหน่วยความจำหลักขณะประมวลผล เมื่อเทียบกับอาร์เรย์ (array)
 - ใช้ตัวชี้ร่วมกับฟังก์ชันเพื่อเพิ่มประสิทธิภาพการเขียนโปรแกรม
 - ตัวแปรชนิดพอยน์เตอร์จะเก็บค่าที่อยู่ของหน่วยความจำหลัก ซึ่งต่างกับตัวแปรปกติที่เก็บค่าที่แท้จริงของข้อมูล
 - การใช้ตัวแปรชนิดพอยน์เตอร์จะเป็นการเข้าถึงข้อมูลหรือเป็นการอ้าง ถึงตำแหน่งที่เก็บข้อมูล
 - การประกาศตัวแปรประเภทพอยน์เตอร์
- `int *ip;` เป็นการประกาศตัวแปร ip ให้เป็นตัวแปรพอยน์เตอร์ที่ชี้ไปยังตัวแปรประเภท int
- `double *dp;` เป็นตัวแปรพอยน์เตอร์ที่ชี้ไปยังตัวแปรประเภท double

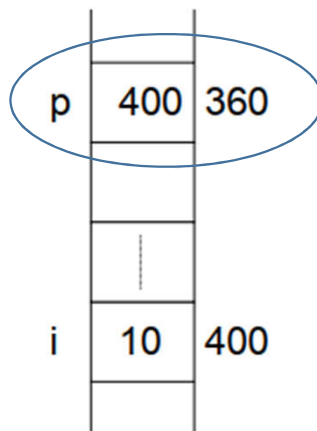
ภาพจำลองการแทนข้อมูลในหน่วยความจำแบบปกติ

```
int i;  
i = 10;
```



```
int i = 10;  
int *p = &i;
```

```
cout<<i<<endl;  
cout<<p<<endl;  
cout<<&i<<endl;  
cout<<*p<<endl;
```



ตัวแปร pointer

`int *p = &i;` คือ `p` เก็บค่าของตำแหน่ง คือ `400`
และในช่อง `400` หรือ `i` เก็บ ค่าจริงๆ คือ `10`

การประกาศตัวแปรประเภทตัวชี้

```
int x = 1, y = 2;
```

```
int *ip, *iq;
```

* คือ ค่า

ตัวมัน คือ address

```
ip = &x;
```

```
y = *ip;
```

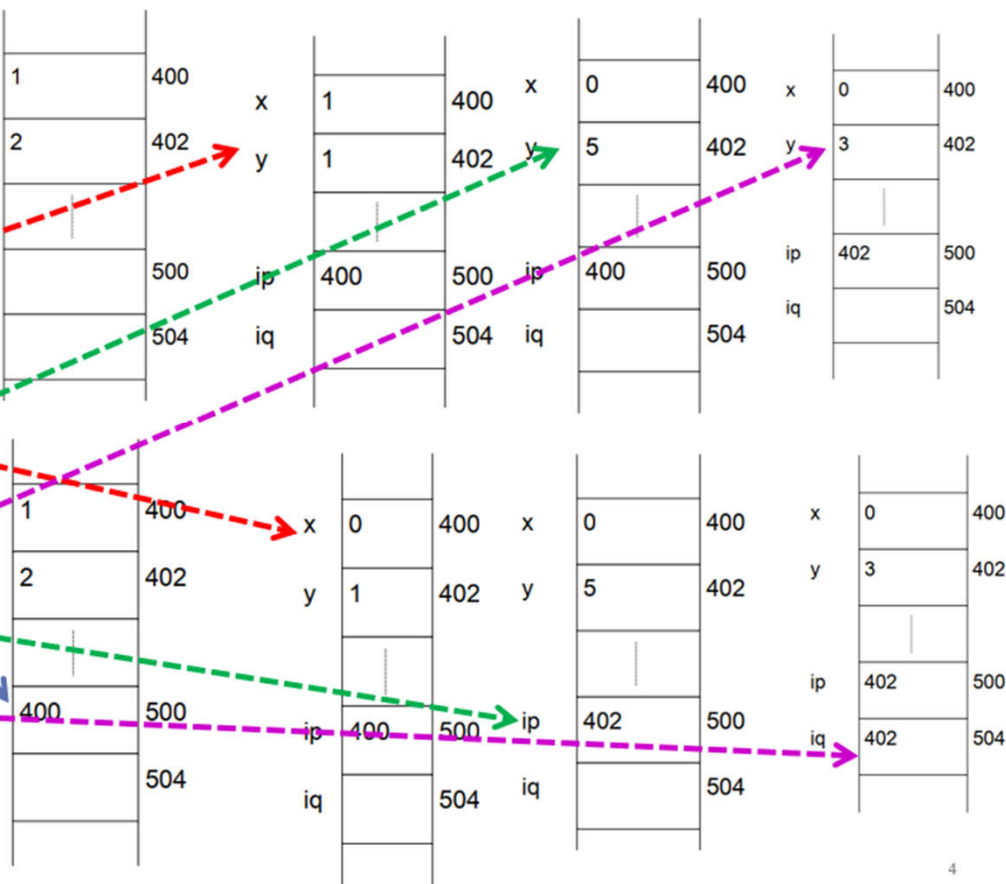
```
*ip = 0;
```

```
y = 5;
```

```
ip = &y;
```

```
*ip = 3;
```

```
iq = ip;
```



ตัวชี้และอาร์กิวเมนต์ของฟังก์ชัน

- เนื่องจากภาษาซีมีการส่งอาร์กิวเมนต์ให้กับฟังก์ชันแบบ By Value และฟังก์ชันสามารถคืนค่า (return) **ค่าได้เพียงหนึ่งค่า**หากต้องการให้ฟังก์ชันมีการเปลี่ยนแปลงค่าและคืนค่ากลับมายัง **ฟังก์ชันที่เรียกใช้** **มากกว่าหนึ่งค่า**จะต้องนำพอยน์เตอร์เข้ามาช่วย
- อาร์กิวเมนต์ที่เป็นประเภทพอยน์เตอร์จะช่วยให้ **ฟังก์ชันสามารถเปลี่ยนค่าให้กับตัวแปร**ที่ส่งเข้ามาได้ เนื่องจากอาร์กิวเมนต์นั้นจะเก็บแอดเดรสของตัวแปรที่ส่งเข้ามา เมื่อมีการเปลี่ยนแปลงค่าของ **อาร์กิวเมนต์ผ่าน reference** ค่าของตัวแปรที่ส่งเข้ามาจะถูกเปลี่ยนค่าพร้อมกันในทันที
- จากตัวอย่างนี้ เราไม่ต้อง return แต่เราสามารถกำหนดค่าให้ตัวแปรได้เลย คือตัวแปร x กับ y โดยอาศัยการแก้ค่าโดย pass by reference

```
void swap (int *px, int *py)
{
    int temp;
    temp = *px;
    *px = *py;
    *py = temp;
}

int main ( )
{
    int x = 5, y = 10;
    printf("Before swap : x = %d", x, ",y = %d\n", y);
    swap ( &x, &y);
    printf("After swap : x = %d", x, ", y = %d\n", y);
}
```

ตัวชี้กับอาร์เรย์

- อาร์เรย์เป็นประเภทข้อมูลที่เก็บชุดของข้อมูลประเภทเดียวกัน มักใช้กับการทำงานที่ต้องทำงานกับตัวแปรชนิดเดียวกันหลายตัวที่มี การทำงานเหมือนกัน เช่น คะแนนของนักศึกษาภายในห้อง 20 คน เป็นต้น อาร์เรย์ในภาษาซีจะนำหลักการของพอยน์เตอร์เข้ามาใช้
- การทำงานใด ๆ ของอาร์เรย์สามารถใช้พอยน์เตอร์เข้ามาแทนที่
- อาร์เรย์ = pointer

```
int v[4]; char s1[2]; float f[10];
```

```
v[0] = 10; Start at 0
```

```
v[3] = 15; END
```

```
v[4] = 5; Error ค่าที่ผิดไป การอ้างข้อมูลต้องอยู่ในขอบเขต
```



ให้อ่านค่าของจำนวนเต็ม 5 จำนวนจากคีย์บอร์ด และแสดงผลในลำดับที่กลับกัน

```
# include <stdio.h>
# define SIZE 5
main ( )
{
    int table[SIZE];
    for (int k = 0; k < SIZE; k++)
        scanf ("%d", &table[k]);
    for (int k = SIZE-1; k >= 0; k--)
        printf ("%d\n", table[k]);
}
```



การใช้ตัวชี้กับอาร์เรย์

```
int a[10];
```

```
int *pa;
```

กำหนดให้พอยน์เตอร์ pa ชี้ไปยังอาร์เรย์ a ด้วยคำสั่ง

```
pa = &a[0];
```

หรือ

```
pa = a;
```

pa จะเก็บค่าแอดเดรสเริ่มต้นของอาร์เรย์ a

```
pa+1;
```

การอ้างอิงแอดเดรสของ a[1]

```
*(pa+1);
```

ค่าของ a[1]

```
char *s;
```

```
char s[];
```

f (&a[2]) หรือ f (a+2) เป็นการส่ง แอดเดรสของสมาชิก a[2]
ให้กับฟังก์ชัน f สามารถทำได้โดยการประกาศ

```
f (int arr[]) { ..... }
```

```
f (int *arr) { ..... }
```


ตัวชี้ตัวอักษรและฟังก์ชัน

```
int strlen (char *s)
{
    char *p = s;
    while (*p != '\0')
        p++;
    return p-s;
}
```

เนื่องจาก s ชี้อยู่ที่ตำแหน่งเริ่มต้น โดยมี p ชีไปที่ s เช่นเดียวกัน แต่จะมีการเลื่อน p ไปทีละหนึ่งตำแหน่ง จนกว่าค่าที่ตำแหน่งที่ p ชี้อยู่จะเท่ากับ '\0' เมื่อนำ p ค่าสุดท้ายมาลบกับ s ที่ตำแหน่ง เริ่มต้นก็จะได้ ความยาวของข้อมูลที่ส่งเข้ามา



- pmesssage
- | | | | | | | | | | | | | | |
|--|---|---|---|---|---|---|--|---|---|---|---|---|---|
| | H | e | l | l | o | , | | w | o | r | l | d | \ |
|--|---|---|---|---|---|---|--|---|---|---|---|---|---|
- amesssage
- | | | | | | | | | | | | | |
|---|---|---|---|---|---|--|---|---|---|---|---|---|
| H | e | l | l | o | , | | w | o | r | l | d | \ |
|---|---|---|---|---|---|--|---|---|---|---|---|---|



ตัวชี้ตัวอักษรและฟังก์ชัน

strcpy () ➔ Array

```
void strcpy ( char *s, char *t )  
{  
    int i=0;  
    while( ( s[i] = t[i] ) != '\0' )  
        i++;  
}
```

strcpy () ➔ Point

```
void strcpy ( char *s, char *t )  
{  
    while ( ( *s = *t ) != '\0' )  
    {  
        s++;  
        t++;  
    }  
}
```



การประกาศตัวแปรชี้ (pointer) ชี้ไปยัง STRUCT

- กรณีการส่งอากิวเมนต์เป็นตัวแปร struct จะไม่เหมาะกับ struct ที่มีขนาดใหญ่ เนื่องจากทุกครั้งที่ส่งตัวแปร struct จะเป็นการสำเนาตัวแปรตัวใหม่ขึ้นมาในฟังก์ชันซึ่งจะทำให้ช้าและเปลืองพื้นที่หน่วยความจำ เราจะใช้พอยน์เตอร์เข้ามาช่วยแก้ปัญหานี้
- โดยส่งแอดเดรสของตัวแปร struct มายังฟังก์ชันซึ่งรับอากิวเมนต์ เป็นพอยน์เตอร์อากิวเมนต์จะชี้ไปยังแอดเดรสเริ่มต้นของตัวแปร struct จะช่วยให้การทำงานเร็วขึ้นและเปลืองหน่วยความจำน้อยลง แต่สิ่งที่ต้องระวังคือ หากมีการเปลี่ยนแปลงค่าที่อากิวเมนต์พอยน์เตอร์ชี้อยู่ ค่าในตัวแปร struct ที่ส่งมายังฟังก์ชันจะเปลี่ยนตามโดยอัตโนมัติ



การประกาศตัวแปรชี้ (pointer) ชี้ไปยัง STRUCT

```
struct point origin;
```

```
point *pp;
```

```
pp = &origin;
```

```
printf ( "origin is (%d, %d)\n", (*pp).x, (*pp).y );
```

```
printf ( "origin is (%d, %d)\n", pp->x, pp->y);
```

(*pp).x จะไม่เหมือนกับ *pp.x เนื่องจากเครื่องหมาย . จะมีลำดับความสำคัญสูงกว่า *

- การอ้างถึงสมาชิกโครงสร้างโดยใช้เครื่องหมาย ->

```
ptrdate->day = 7;
```

```
scanf ( "%d", &ptrdate->year );
```

- การอ้างถึงสมาชิกของโครงสร้างผ่านตัวแปรพอยน์เตอร์

```
(*ptrdate).day = 7;
```

```
scanf ( "%d", &((*ptrdate).year) );
```

การประกาศตัวแปรชี้ (pointer) ชี้ไปยัง STRUCT

```
typedef struct {
    int day;
    int month;
    int year;
} Date;

Date today;

Date *ptrdate;
```

แบบที่ 1

การประกาศแบบ
ข้อมูลโครงสร้าง

การประกาศตัวแปร
ข้อมูลแบบโครงสร้าง

การประกาศตัวแปร
pointer ชี้ไปยังโครงสร้าง

```
struct date {
    int day;
    int month;
    int year;
} *ptrdate;
```

แบบที่ 2

```
typedef struct {
    int day;
    int month;
    int year;
} Date;

typedef Date *PtrDate;

PtrDate ptrdate;
```

แบบที่ 3

การประกาศแบบ
ข้อมูลโครงสร้าง

การประกาศประเภท
ตัวแปร pointer ชี้ไปยัง
โครงสร้าง

การประกาศตัวแปร
pointer ชี้ไปยัง โครงสร้าง



การประกาศตัวแปรชี้ (pointer) ชี้ไปยัง STRUCT

```
struct date
{
    int day;
    int month;
    int year;
};
main ( )
{
    struct date    today;
    struct date    *ptrdate;

    ptrdate = &today;
    ptrdate->day = 27;
    ptrdate->month = 9;
    ptrdate->year = 1985;
    printf ( "Today\'s date is %2d/%2d/%4d\n", ptrdate->day, ptrdate->month, ptrdate->year );
}
```



การใช้งาน Pointer

```
#include<stdio.h>
void main( )
{
    int N = 100;
    int *pN;
    pN = &N;

    printf ( "%p\n", &N );
    printf ( "%p\n", pN );

    printf ( "%d\n", N );
    printf ( "%d\n", *pN );
}
```

ประกาศ
เชื่อม

ตัวมันเป็น address

ค่าของมัน



การใช้งาน Pointer of FUNCTION

```
#include<stdio.h>
void sum_value( int a, int b) //pass by value
{
    a = 3;
    b = 4;
    printf( "%d %d\n", a, b );
}
```

```
void sum_ref( int *a, int *b) //pass by reference
```

```
{
    *a = 5;
    *b = 6;
    printf( "%d %d\n", *a, *b );
}
```

```
void main( )
```

```
{
    int x = 1;
    int y = 2;
    printf( "%d %d\n",x, y );
    sum_value(x,y);
    printf( "%d %d\n",x, y );
    sum_ref(&x,&y);
    printf( "%d %d\n",x, y );
}
```

1 กับ 2

3 กับ 4

เหมือนเดิม 1 กับ 2

ส่ง address ให้ x กับ y คือ 5 กับ 6

x กับ y คือ 5 กับ 6



การใช้งาน Pointer of Array

```
#include<stdio.h>
```

```
void main( )
```

```
{
```

```
    int x[] = {1,2,3,4,5};
```

```
    printf( "%p\n", x);
```

```
    int *y = x;
```

```
    printf( "%p\n", y);
```

```
    printf( "%d\n", x[0]);
```

```
    printf( "%d\n", *(y+0) );
```

```
    printf( "%d\n", x[1]);
```

```
    printf( "%d\n", *(y+1) );
```

```
}
```

x คือ pointer และ x คือ Array

อ้างอิงตำแหน่ง

อ้างอิงตำแหน่ง

อ้างอิงตำแหน่ง

อ้างอิงตำแหน่ง



การใช้งาน Pointer of String

```
char *s1 = "my name is be1";
printf( "%s\n", s1);
printf( "%c\n", *(s1+1) );
char s2[] = "my name is be2";
printf( "%s\n", s2);
printf( "%c\n", s2[3] );
s1 = "I love U";
//*(s1+2) = 'x';
s2[2] = 'x';
printf( "%s\n", s2);
char *s3[] = {"s1","s2","s3"};
printf( "%s\n", s3[0] );
s3[1] = "GOD";
printf( "%s\n", s3[1] );
char *s4 = s2;
s4[2] = 'k';
printf( "%s\n", s2);
printf( "%s\n", s4);
```

ข้อความเป็น pointer

แสดงทั้งหมด

เอาค่าตัวเดียว

ข้อความเป็น []

เอาค่าตัวเดียว

เปลี่ยนทั้งอันทำได้

//Error

ทำได้

ข้อความเป็น pointe + array

การ copy string ต้องระวัง เพราะถ้าเขียน char *s4 = s1;
ถ้าต้นฉบับเป็น Array สามารถแก้ไขค่าได้
s1 กับ s4 เชื่อมกัน ถ้าเปลี่ยน เปลี่ยนทั้งคู่



การใช้งาน Pointer of String

```
char *s1 = "test";  
char s2[4];  
char s3[4];  
char *s4 = s3;
```

```
printf( "%s\n", s1);
```

สามารถทำได้

```
scanf("%s",s1);  
printf( "%s\n", s1);
```

ไม่สามารถทำได้ ถ้าเป็น **pointer**
สามารถทำได้

```
scanf("%s",s2);  
printf( "%s\n", s2);
```

สามารถทำได้ เป็น **Array** สามารถทำได้
สามารถทำได้

```
scanf("%s",s4);  
printf( "%s\n", s4);
```

สามารถทำได้ ถ้ากรณีเป็น **Array** มาก่อน
สามารถทำได้



การใช้งาน Pointer of String

```
char name[100][100];  
scanf("%s",name[i]);
```

กรณีใช้ String Array ควรใช้ Array 2 มิติ
อ้างอิง Array 2 มิติ

```
int num[100];  
scanf("%d",&num[i]);
```

หรืออาจจะใช้ pointer ก็ได้

ตัวอย่างการ Copy String และ Compare String

```
#include<string.h>
```

```
char str1[20] = "C programming";  
char str2[20];  
strcpy(str2, str1);
```

```
char str1[20] = "C programming";  
char str2[20] = "C programming";  
if( strcmp( id[i], pass ) == 0 )  
{  
}
```



การใช้งาน Pointer of STRUCT

```
#include<stdio.h>
```

```
#include<string.h>
```

```
struct Books
```

```
{
```

```
    char title[50];
```

```
    int book_id;
```

```
};
```

```
void printBook( struct Books *book )
```

```
{
```

```
    printf( "Book title : %s\n", book->title);
```

```
    printf( "Book book_id : %d\n", book->book_id);
```

```
}
```

```
void main( )
```

```
{
```

```
    struct Books Book1;
```

```
    strcpy( Book1.title, "C Programming");
```

```
    Book1.book_id = 6495407;
```

```
    printf( "Book 1 title : %s\n", Book1.title);
```

```
    printf( "Book 1 book_id : %d\n", Book1.book_id);
```

```
    printBook( &Book1 );
```

```
}
```

ควร import string เพราะต้องใช้สำหรับการใส่ค่า

struct ใส่ function ข้างในไม่ได้ จึงต้อง pass parameter โดย pointer

รับค่าเป็น pointer

ถ้ากรณีอ้างอิงผ่าน pointer ใช้ ->

สร้าง structure

พยายามใช้ฟังก์ชันสำเร็จรูป

ถ้ากรณีปกติใช้ "."